



Universidade Estadual de Santa Cruz – UESC

Relatório de Implementações de Métodos da Disciplina Análise Numérica

**Relatório de implementações
realizadas por Davi Roriz Oliveira**

Disciplina: Análise Numérica

Curso: Ciência da Computação

Semestre 2024.1

Professor: Gesil Sampaio Amarante II

**Ilhéus – BA
2024**

Índice

1. Linguagem escolhida e justificativas.....	3
2. MMQ - Regressão Linear.....	4
3. MMQ - Caso Discreto.....	8
4. MMQ - Caso Contínuo.....	12
5. Polinômio de Lagrange.....	16
6. Polinômio de Diferença Dividida de Newton.....	20
7. Derivadas de Primeira Ordem.....	24
8. Derivadas de Segunda Ordem.....	27
9. Trapézio Simples.....	30
10. Trapézio Múltiplo.....	32
11. Simpson 1/3.....	35
12. Simpson 3/8.....	38
13. Extrapolação por Richards.....	41
14. Quadratura de Gauss.....	44
15. Conclusões Finais.....	46

1. Linguagem escolhida e justificativas

Para a realização do projeto, foi utilizado a linguagem de programação Python na sua versão 3.11.2, sua escolha deve-se ao fato da facilidade na implementação de códigos para cálculos matemáticos. Pois, a linguagem possui uma ampla variedade de bibliotecas que auxiliam a codificação de algoritmos com estes.

As bibliotecas Sympy, Math e Numpy, que foram utilizadas no projeto, tem como objetivo justamente fornecer ferramentas de álgebra computacional para os programas. Também temos a biblioteca Os, que foi utilizada para acessar o diretório em que os arquivos se encontram e poder abrir os arquivos de entrada e saída. Por fim, Matplotlib que foi necessária para plotar os gráficos fornecidos no projeto.

Além de serem de fácil instalação, a linguagem também é multiplataforma, podendo operar em diferentes sistemas operacionais, bem como Windows, Linux e MacOS. Sua sintaxe é simples comparada a outras linguagens, possibilitando que programadores de diferentes níveis de conhecimento possam aprender e manipular códigos em Python, bem como compreender as implementações dos códigos deste projeto.

2. MMQ - Regressão Linear

Estratégia de Implementação:

O arquivo contém 3 funções principais: `main()`, `grafico()` e `regressaoLinear()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `regressaoLinear()` tem como objetivo fazer o cálculo do método e gerar o polinômio que mais se aproxima do conjunto de dados. Ela tem como parâmetros dois vetores: `PontosX` e `PontosY`, que contém o conjunto de dados a serem analisados, e retorna o polinômio $f(x)$.

Por último, a função `grafico()` recebe os vetores `PontosX` e `PontosY`, além da função $f(x)$ gerada pelo método e plota o gráfico que contém o conjunto de dados e a função $f(x)$. Para a realização do gráfico, foi necessário utilizar as bibliotecas `Matplotlib` e `Numpy`.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em duas linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço. Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em uma única linha para cada caso teste, contendo o polinômio $f(x)$ gerado.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro *Cálculo Numérico* Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado polinômio que mais se aproxima dos pontos.

No gráfico, os pontos do conjunto de dados estão na cor vermelha, enquanto o polinômio gerado está na cor azul.

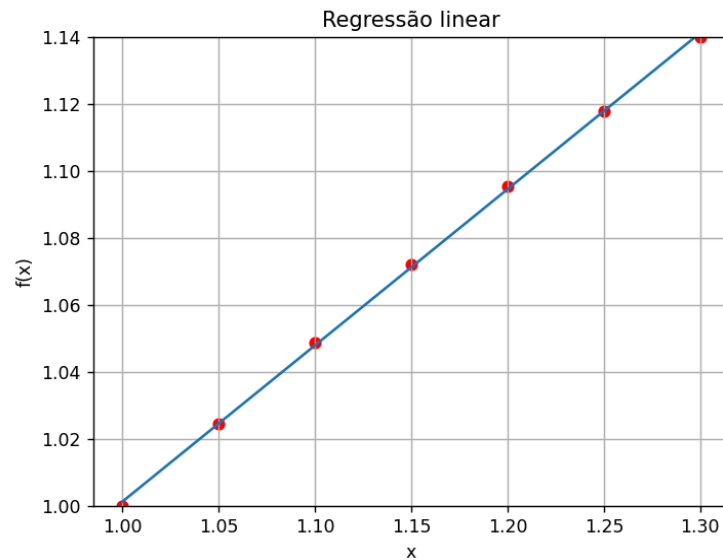
1. Problema 11.1:

Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Resultado obtido: $f(x) = 0.4668 * x + 0.5345$



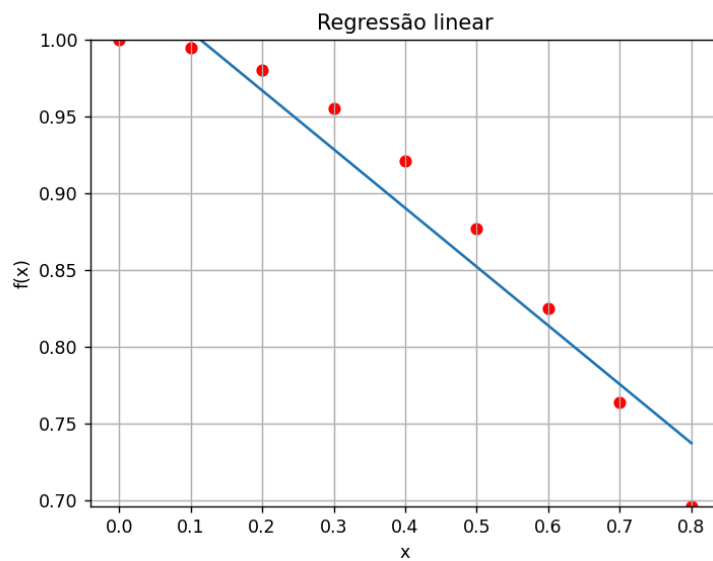
2. Problema 11.2:

Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Resultado Obtido: $f(x) = -0.3828 * x + 1.0435$



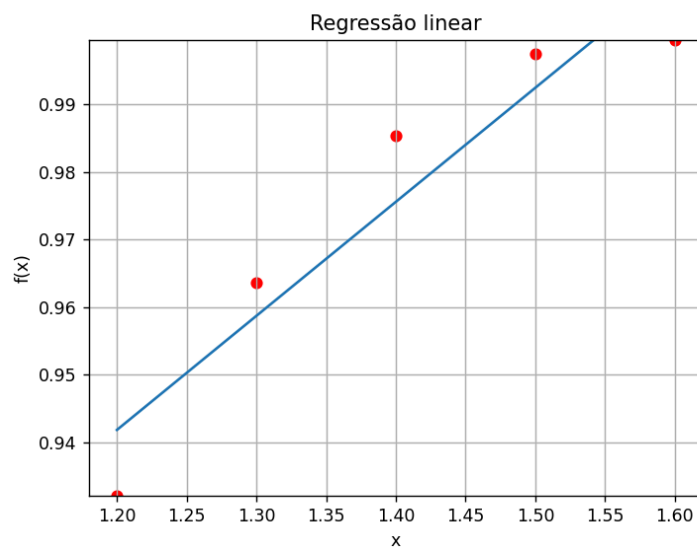
3. Problema 11.3:

Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Resultado Obtido: $f(x) = 0.169 * x + 0.739$



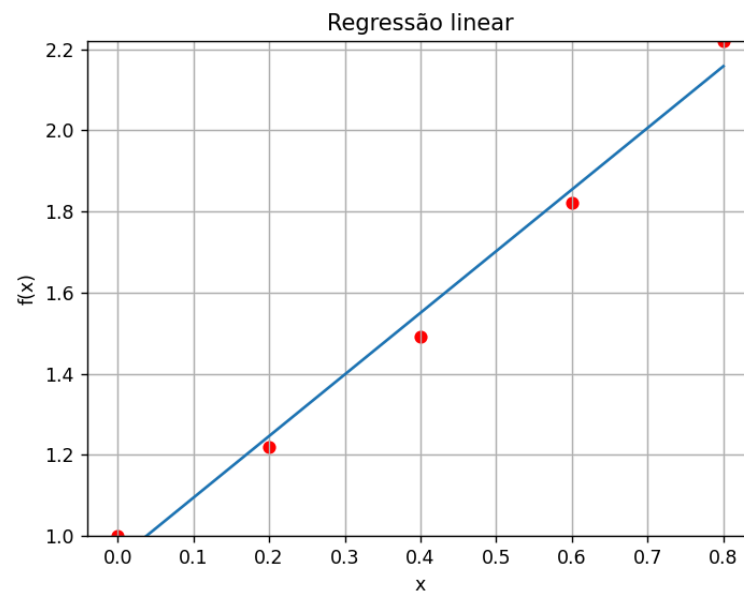
4. Problema 11.4:

Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Resultado Obtido: $f(x) = 1.52 * x + 0.942$



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

3. MMQ - Caso Discreto

Estratégia de Implementação:

O arquivo contém 3 funções principais: `main()`, `grafico()` e `discreto()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `discreto()`, tem como objetivo fazer o cálculo do método e gerar o polinômio que mais se aproxima do conjunto de dados. `discreto()` tem como parâmetros dois vetores: `PontosX` e `PontosY`, que contém o conjunto de dados a serem analisados, e o grau do polinômio que vai ser gerado e retorna o polinômio $f(x)$.

Por último, a função `grafico()` recebe os vetores `PontosX` e `PontosY`, além da função $f(x)$ gerada pelo método e plota o gráfico que contém o conjunto de dados e a função $f(x)$. Para a realização do gráfico, foi necessário utilizar as bibliotecas Matplotlib e Numpy.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço e, por fim, uma linha contendo o grau do polinômio que vai ser gerado. Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em uma única linha para cada caso teste, contendo o polinômio $f(x)$ gerado.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado polinômio que mais se aproxima dos pontos.

No gráfico, os pontos do conjunto de dados estão na cor vermelha, enquanto o polinômio gerado está na cor azul.

1. Problema 11.1:

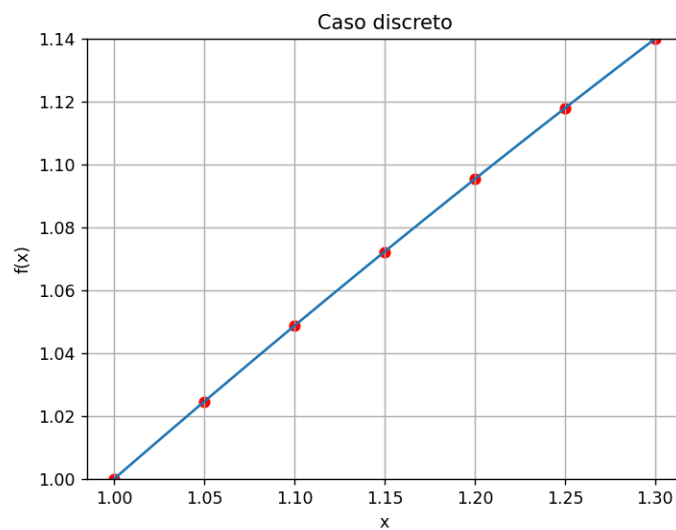
Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Grau: 2

Resultado obtido: $f(x) = -0.101428571428575 \cdot x^2 + 0.700071428571438 \cdot x + 0.401399999999995$



2. Problema 11.2:

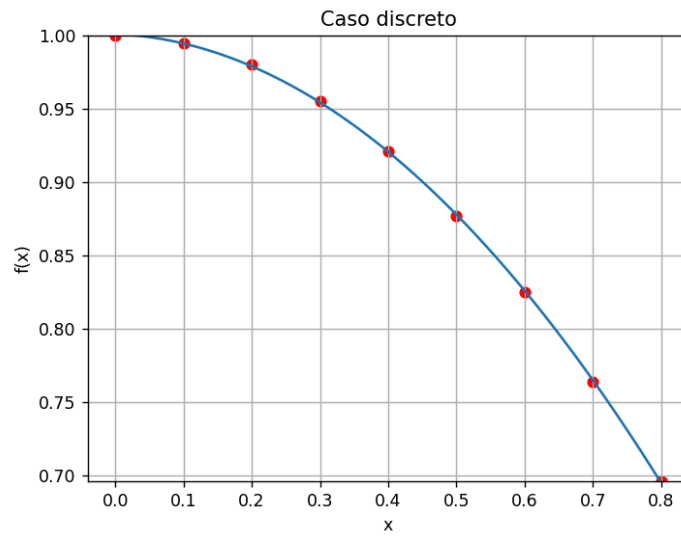
Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Grau: 2

Resultado Obtido: $f(x) = -0.45487012987013 \cdot x^2 - 0.0189372294372288 \cdot x + 1.00101212121212$



3. Problema 11.3:

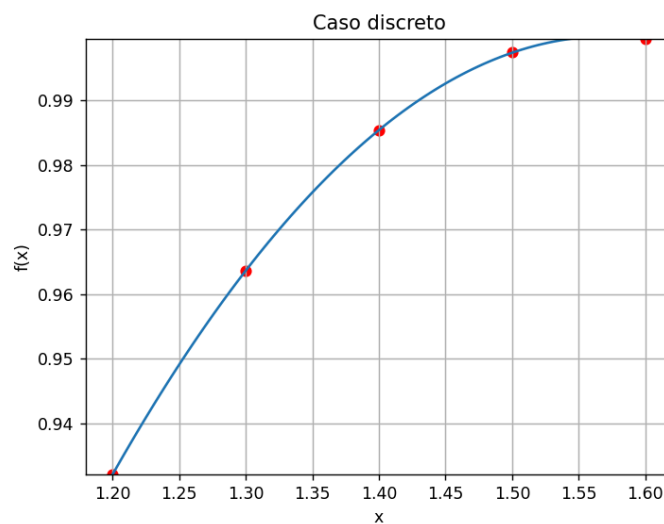
Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Grau: 2

Resultado Obtido: $f(x) = -0.490928571428566 \cdot x^2 + 1.54358999999998 \cdot x - 0.213365428571414$



4. Problema 11.4:

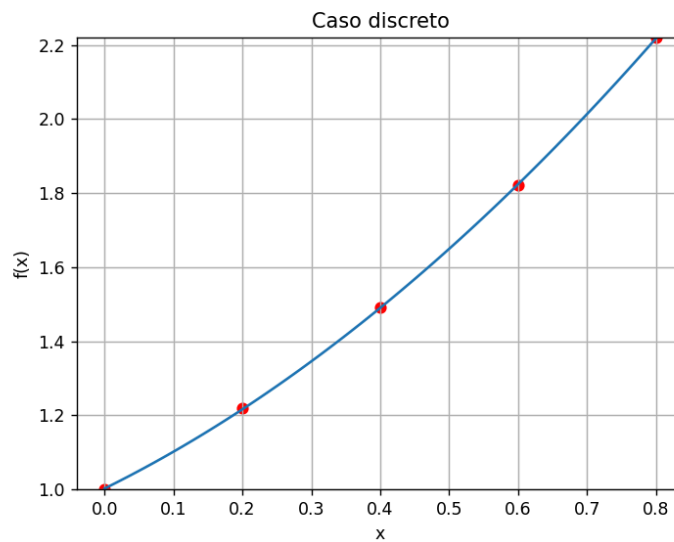
Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Grau: 2

Resultado Obtido: $f(x) = 0.7499999999999999 * x^2 + 0.9200000000000001 * x + 1.002$



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

4. MMQ - Caso Contínuo

Estratégia de Implementação:

O arquivo contém 3 funções principais: `main()`, `grafico()` e `continuo()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `continuo()`, tem como objetivo fazer o cálculo do método e gerar o polinômio que mais se aproxima do conjunto de dados. `continuo()` tem como parâmetros dois vetores: `PontosX` e `PontosY`, que contém o conjunto de dados a serem analisados, e o grau do polinômio que vai ser gerado e retorna o polinômio $f(x)$.

Por último, a função `grafico()` recebe os vetores `PontosX` e `PontosY`, além da função $f(x)$ gerada pelo método e plota o gráfico que contém o conjunto de dados e a função $f(x)$. Para a realização do gráfico, foi necessário utilizar as bibliotecas Matplotlib e Numpy.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço e, por fim, uma linha contendo o grau do polinômio que vai ser gerado. Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em uma única linha para cada caso teste, contendo o polinômio $f(x)$ gerado.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado polinômio que mais se aproxima dos pontos.

No gráfico, os pontos do conjunto de dados estão na cor vermelha, enquanto o polinômio gerado está na cor azul.

1. Problema 11.1:

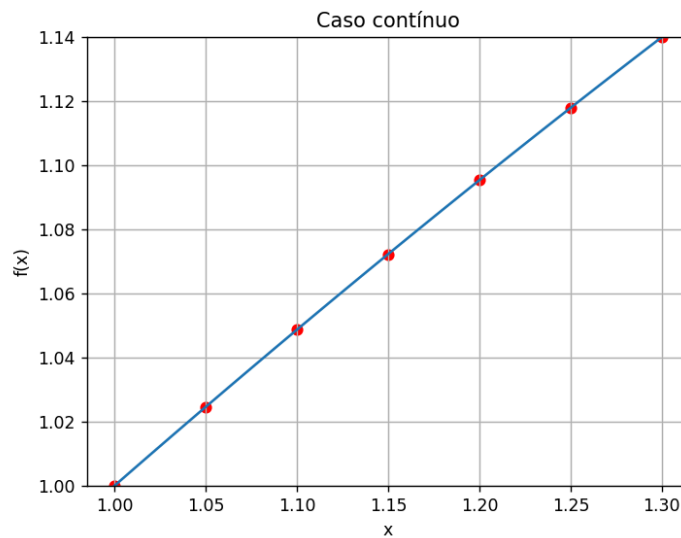
Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Grau: 2

Resultado obtido: $f(x) = -0.101428571427611 * x^{**2} + 0.700071428569227 * x + 0.401400000001252$



2. Problema 11.2:

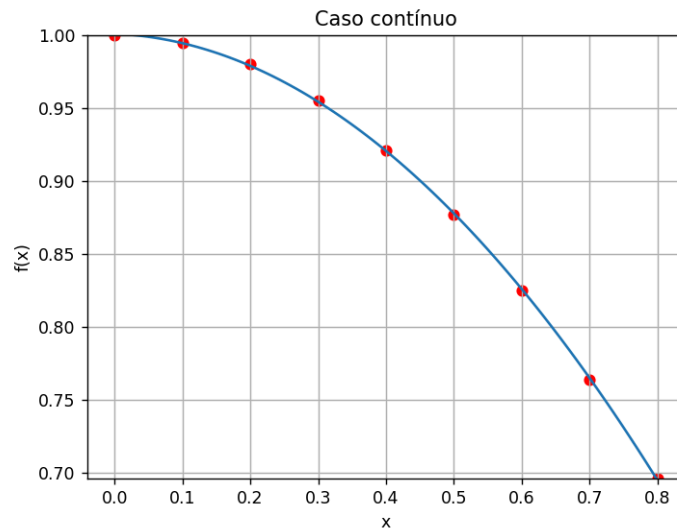
Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Grau: 2

Resultado Obtido: $f(x) = -0.454870129870137 * x^{**2} - 0.0189372294372225 * x + 1.00101212121212$



3. Problema 11.3:

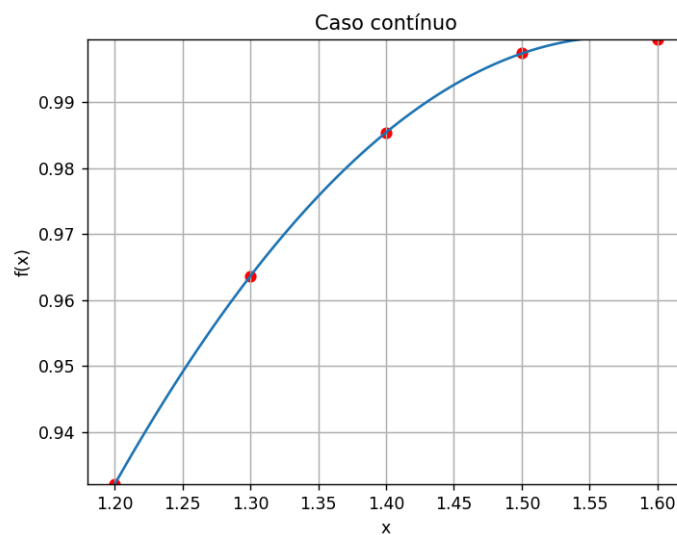
Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Grau: 2

Resultado Obtido: $f(x) = -0.490928571429776 \cdot x^2 + 1.54359000000337 \cdot x - 0.213365428573762$



4. Problema 11.4:

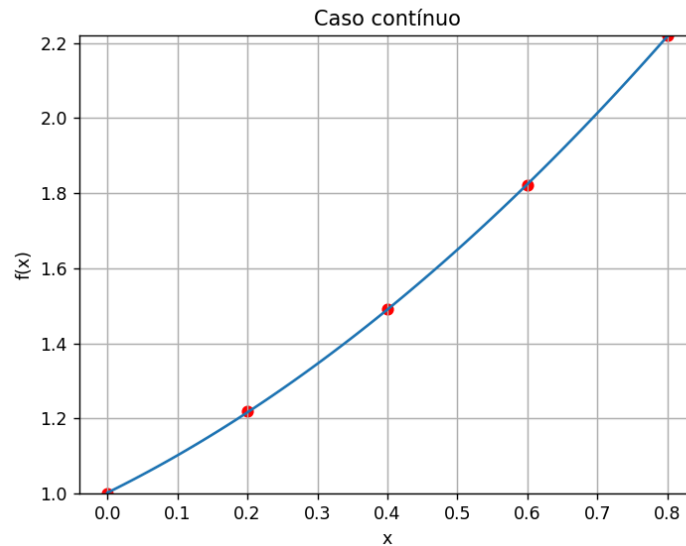
Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Grau: 2

Resultado Obtido: $f(x) = 0.750000000000032 * x^2 + 0.919999999999974 * x + 1.002$



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

5. Polinômio de Lagrange

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `lagrange()` e `interpolador()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

As funções `lagrange()` e `interpolador()`, tem como objetivo fazer o cálculo do método e gerar o polinômio que mais se aproxima do conjunto de dados. `lagrange()` tem como parâmetros dois vetores: `PontosX` e `PontosY`, que contém o conjunto de dados a serem analisados, faz a chamada da função `interpolador()`, e retorna o polinômio $f(x)$.

Por último, a função `grafico()` recebe os vetores `PontosX` e `PontosY`, além da função $f(x)$ gerada pelo método e plota o gráfico que contém o conjunto de dados e a função $f(x)$. Para a realização do gráfico, foi necessário utilizar as bibliotecas Matplotlib e Numpy.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em duas linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço. Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em uma única linha para cada caso teste, contendo o polinômio $f(x)$ gerado.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado polinômio que mais se aproxima dos pontos.

No gráfico, os pontos do conjunto de dados estão na cor vermelha, enquanto o polinômio gerado está na cor azul.

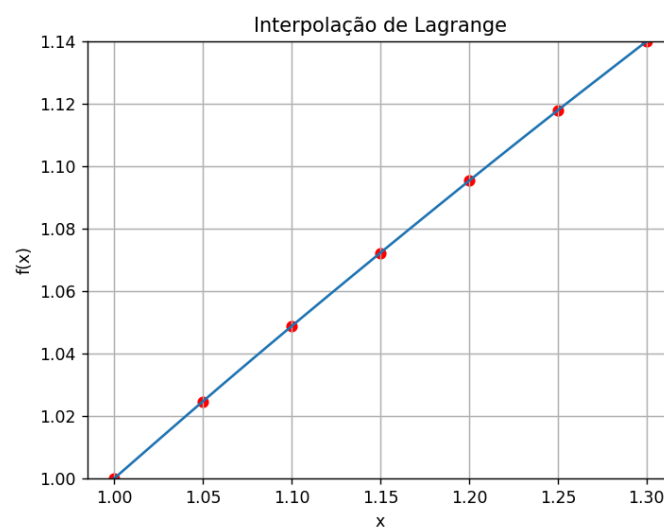
1. Problema 11.1:

Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Resultado obtido: $f(x) = 79.999999998108x^{**6} - 553.333333329414x^{**5} + 1591.66666665813x^{**4} - 2437.14999999059x^{**3} + 2094.9203333254x^{**2} - 957.89666665555x + 182.792999998201$



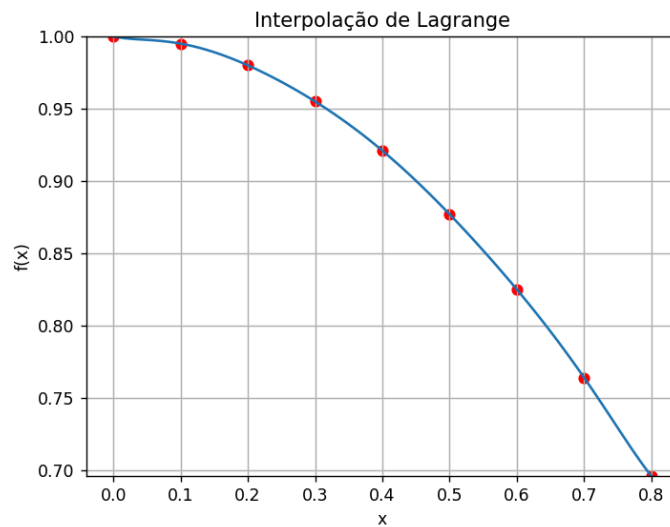
2. Problema 11.2:

Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Resultado Obtido: $f(x) = 104.166666666642x^{**8} - 329.365079364926x^{**7} + 425.694444444399x^{**6} - 289.305555555554x^{**5} + 110.579861111112x^{**4} - 23.426388888885x^{**3} + 2.00152777777868x^{**2} - 0.101476190476171x + 1.0$



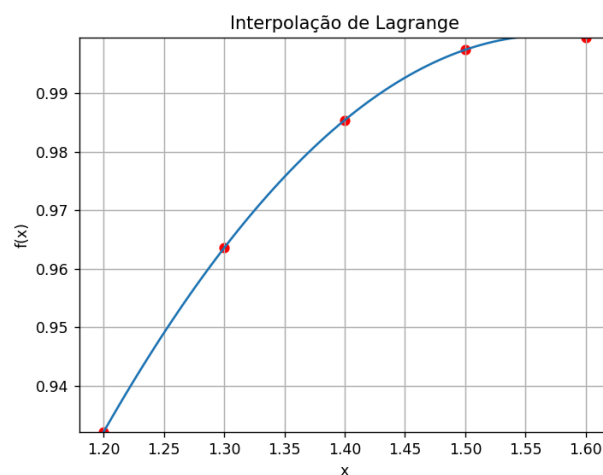
3. Problema 11.3:

Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Resultado Obtido: $f(x) = 0.045833333333178 \cdot x^4 - 0.284166666661804 \cdot x^3 + 0.161541666663652 \cdot x^2 + 0.885441666667248 \cdot x + 0.0328899999988153$



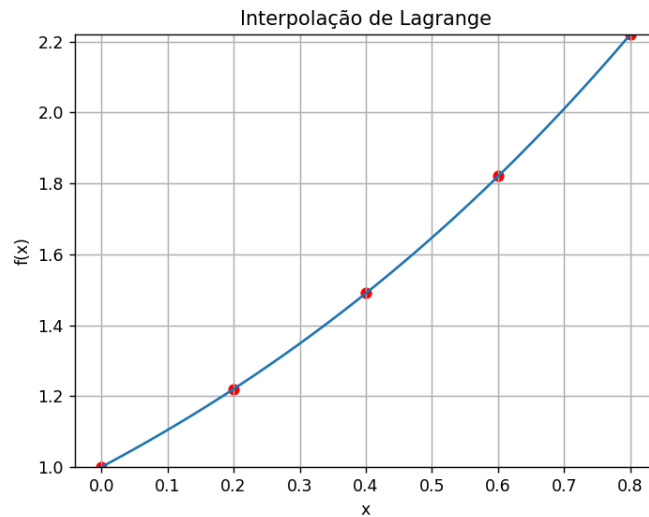
4. Problema 11.4:

Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Resultado Obtido: $f(x) = 2.8421709430404e-14*x**4 + 0.20833333333258*x**3 + 0.5000000000000032*x**2 + 0.991666666666665*x + 1.0$



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

6. Polinômio de Diferença Dividida de Newton

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `newton()` e `interpolador()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

As funções `newton()` e `interpolador()`, tem como objetivo fazer o cálculo do método e gerar o polinômio que mais se aproxima do conjunto de dados. `lagrange()` tem como parâmetros dois vetores: `PontosX` e `PontosY`, que contém o conjunto de dados a serem analisados, faz a chamada da função `interpolador()`, e retorna o polinômio $f(x)$.

Por último, a função `grafico()` recebe os vetores `PontosX` e `PontosY`, além da função $f(x)$ gerada pelo método e plota o gráfico que contém o conjunto de dados e a função $f(x)$. Para a realização do gráfico, foi necessário utilizar as bibliotecas `Matplotlib` e `Numpy`.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em duas linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço. Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em uma única linha para cada caso teste, contendo o polinômio $f(x)$ gerado.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro *Cálculo Numérico* Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado polinômio que mais se aproxima dos pontos.

No gráfico, os pontos do conjunto de dados estão na cor vermelha, enquanto o polinômio gerado está na cor azul.

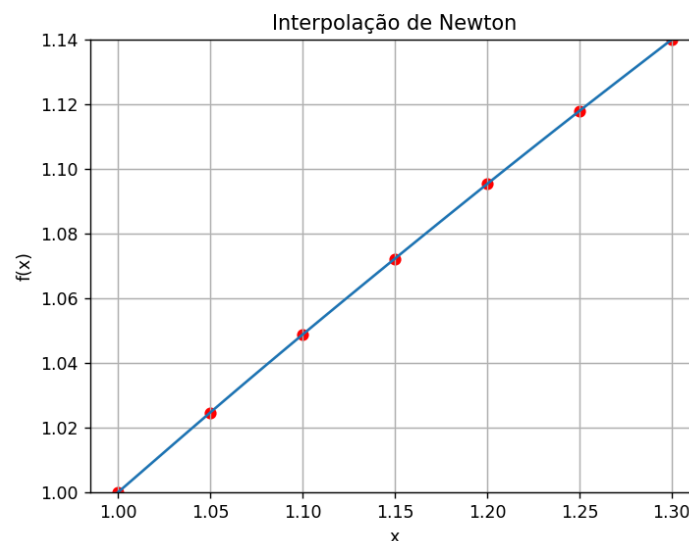
1. Problema 11.1:

Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Resultado obtido: $f(x) = 79.999999999621x^6 - 553.33333333101x^5 + 1591.66666666002x^4 - 2437.14999998989x^3 + 2094.92033332469x^2 - 957.896666662732x + 182.792999999255$



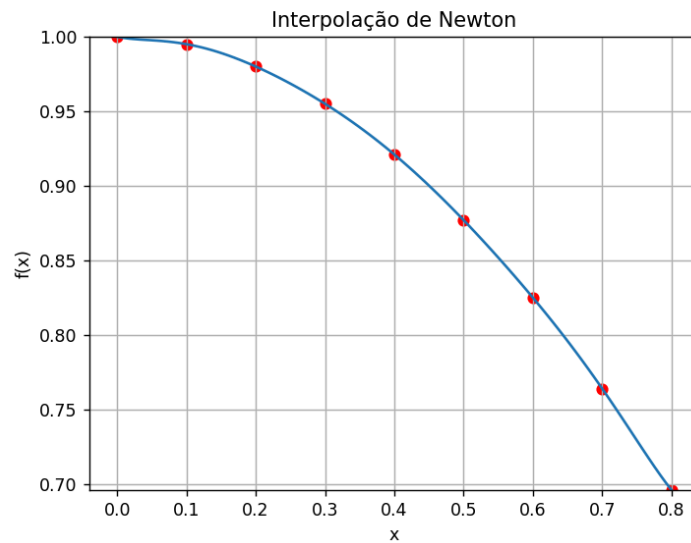
2. Problema 11.2:

Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Resultado Obtido: $f(x) = 104.166666666676x^8 - 329.365079365112x^7 + 425.694444444448x^6 - 289.305555555587x^5 + 110.579861111123x^4 - 23.4263888888916x^3 + 2.00152777777807x^2 - 0.101476190476203x + 1.0$



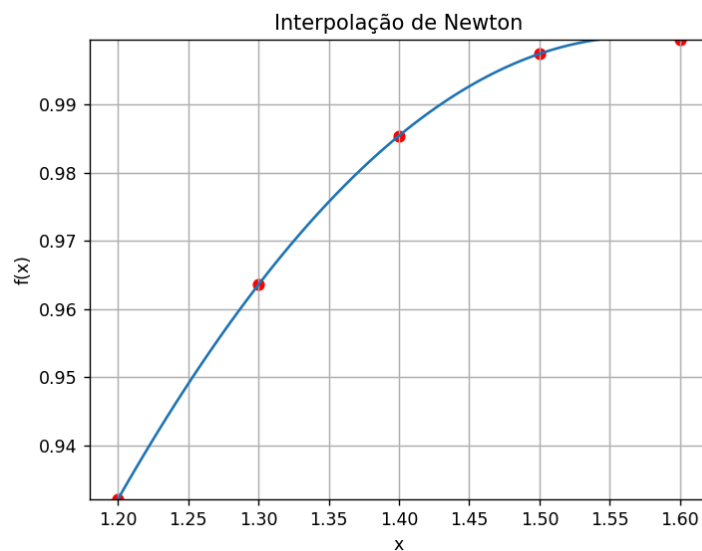
3. Problema 11.3:

Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Resultado Obtido: $f(x) = 0.0458333333335623 \cdot x^4 - 0.284166666667957 \cdot x^3 + 0.16154166666938 \cdot x^2 + 0.885441666664141 \cdot x + 0.032890000000878$



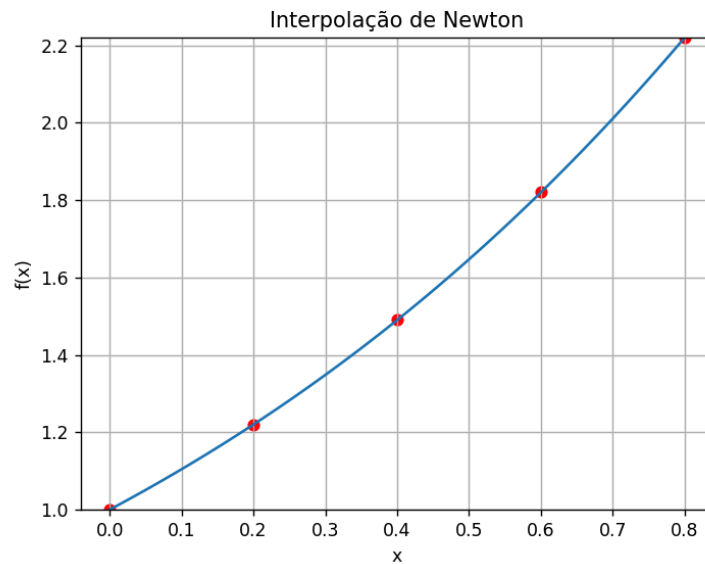
4. Problema 11.4:

Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Resultado Obtido: $f(x) = -1.09287578986539e-14*x**4 + 0.208333333333349*x**3 + 0.499999999999994*x**2 + 0.991666666666667*x + 1.0$



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

7. Derivadas de primeira ordem

Estratégia de Implementação:

O arquivo contém 3 funções que calculam as 3 estimativas das derivadas (Centrada, progressiva e retardada), a função `main()`, e uma versão modificada das funções de polinômios de newton que tem como objetivo calcular o polinômio e retornar sua derivada no ponto determinado.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço e, por fim, uma linha contendo o index do ponto a ser calculado (o primeiro ponto é 0 e o último é N - 1 pontos). Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em quatro linhas, contendo as estimativas das derivadas centrada, progressiva e retardada, além da derivada real.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e geradas as estimativas das derivadas.

1. Problema 11.1:

Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Index: 1

Resultado Obtido:

Derivada centrada: 0.4879999999999991

Derivada progressiva: 0.48199999999999976

Derivada retardada: 0.493999999999999844

Derivada real: 0.4940000000000597

2. Problema 11.2:

Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Index: 1

Resultado Obtido:

Derivada centrada: -0.10000000000000009

Derivada progressiva: -0.15000000000000013

Derivada retardada: -0.050000000000000044

Derivada real: 11.3585238095245

3. Problema 11.3:

Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Index: 1

Resultado Obtido:

Derivada centrada: 0.2670500000000001

Derivada progressiva: 0.21890000000000057

Derivada retardada: 0.31519999999999965

Derivada real: 0.539358333333281

4. Problema 11.4:

Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Index: 1

Resultado Obtido:

Derivada centrada: 1.2249999999999999

Derivada progressiva: 1.35

Derivada retardada: 1.0999999999999999

Derivada real: 2.6166666666666666

Dificuldades enfrentadas:

Em alguns casos, o resultado da derivada real não se mostrou coerente com as estimativas, como por exemplo, o problema 11.2

8. Derivadas de segunda ordem

Estratégia de Implementação:

O arquivo contém 3 funções que calculam as 3 estimativas das derivadas (Centrada, progressiva e retardada), a função `main()`, e uma versão modificada das funções de polinômios de newton que tem como objetivo calcular o polinômio e retornar sua derivada no ponto determinado.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira com as posições do eixo X dos pontos, todos os pontos separados por um espaço e, a segunda linha contendo as posições do eixo Y dos pontos, também separados por um espaço e, por fim, uma linha contendo o index do ponto a ser calculado (o primeiro ponto é 0 e o último é N - 1 pontos). Pode ser posto mais de um caso teste no arquivo.

O arquivo de saída “out.txt” consiste em quatro linhas, contendo as estimativas das derivadas centrada, progressiva e retardada, além da derivada real.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3, 11.4 da página 328 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e geradas as estimativas das derivadas.

1. Problema 11.1:

Entrada:

PontosX: [1, 1.05, 1.1, 1.15, 1.20, 1.25, 1.30]

PontosY: [1, 1.0247, 1.0488, 1.0723, 1.0954, 1.118, 1.1401]

Index: 2

Resultado Obtido:

Derivada centrada: -0.23999999999997315

Derivada progressiva: -0.23999999999997315

Derivada retardada: -0.160000000000007233

Derivada real: 1210.70733332805

2. Problema 11.2:

Entrada:

PontosX: [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

PontosY: [1, 0.995, 0.98, 0.955, 0.921, 0.877, 0.825, 0.764, 0.696]

Index: 2

Resultado Obtido:

Derivada centrada: -0.9999999999999896

Derivada progressiva: -1.0000000000000007

Derivada retardada: -0.8999999999999901

Derivada real: 93741.8308333415

3. Problema 11.3:

Entrada:

PontosX: [1.2, 1.3, 1.4, 1.5, 1.6]

PontosY: [0.93204, 0.96356, 0.98545, 0.99749, 0.99957]

Index: 2

Resultado Obtido:

Derivada centrada: -0.9850000000000162

Derivada progressiva: -0.9629999999999941

Derivada retardada: -0.9959999999999951

Derivada real: -0.886916666665727

4. Problema 11.4:

Entrada:

PontosX: [0, 0.2, 0.4, 0.6, 0.8]

PontosY: [1, 1.22, 1.49, 1.82, 2.22]

Index: 2

Resultado Obtido:

Derivada centrada: 1.5000000000000001

Derivada progressiva: 1.2500000000000009

Derivada retardada: 1.7500000000000024

Derivada real: 3.499999999999965

Dificuldades enfrentadas:

Em alguns casos, o resultado da derivada real não se mostrou coerente com as estimativas, como por exemplo, os problemas 11.1 e 11.2

9. Trapézio simples

Estratégia de Implementação:

O arquivo contém a função `trapezio()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$ e o intervalo proposto nos exemplos, e retorna a estimativa da integral.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: [1.0, 1.3]

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa do trapezio simples: 0.321026313764871

Erro: 0.14%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa do trapezio simples: 0.678682683738866

Erro: 5.39%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa do trapezio simples: 0.386322537801746

Erro: 1.34%

4. Problema 11.11:

Entrada:

Função: $\sin(x) * (E)^{-x}$

Intervalo: $[0.0, 1.2]$

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa do trapezio simples: 0.168434866781561

Erro: 44.79%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

10. Trapézio múltiplo

Estratégia de Implementação:

O arquivo contém a função `trapezio()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$, o intervalo proposto nos exemplos e a quantidade de subdivisões, e retorna a estimativa da integral.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em quatro linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente e a quarta recebe o número de subdivisões.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: $[1.0, 1.3]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa do trapezio multiplo: 0.321472563527613

Erro: 0.0%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Subdivisões: 8

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa do trapezio multiplo: 0.716758194500588

Erro: 0.08%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Subdivisões: 4

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa do trapezio multiplo: 0.391230924651407

Erro: 0.08%

4. Problema 11.11:

Entrada:

Função: $\sin(x) \cdot (E)^{-x}$

Intervalo: $[0.0, 1.2]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa do trapezio multiplo: 0.301165021091142

Erro: 1.28%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

11. Simpson 1/3

Estratégia de Implementação:

O arquivo contém a função `simpson()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$, o intervalo proposto nos exemplos e a quantidade de subdivisões, e retorna a estimativa da integral.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em quatro linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente e a quarta recebe o número de subdivisões.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: $[1.0, 1.3]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa de Simpson 1/3: 0.321485362179150

Erro: 0.0%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Subdivisões: 8

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa de Simpson 1/3: 0.717356489905625

Erro: 0.0%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Subdivisões: 4

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa de Simpson 1/3: 0.391557494569022

Erro: 0.0%

4. Problema 11.11:

Entrada:

Função: $\sin(x) * (E)^{**}(-x)$

Intervalo: $[0.0, 1.2]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa de Simpson 1/3: 0.305056537540836

Erro: 0.0%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

12. Simpson 3/8

Estratégia de Implementação:

O arquivo contém a função `simpson()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$, o intervalo proposto nos exemplos e a quantidade de subdivisões, e retorna a estimativa da integral.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em quatro linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente e a quarta recebe o número de subdivisões.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: $[1.0, 1.3]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa de Simpson 3/8: 0.321485354430579

Erro: 0.0%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Subdivisões: 8

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa de Simpson 3/8: 0.786144018913135

Erro: -9.59%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Subdivisões: 4

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa de Simpson 3/8: 0.291698866320963

Erro: 25.50%

4. Problema 11.11:

Entrada:

Função: $\sin(x) * (E)^{**}(-x)$

Intervalo: $[0.0, 1.2]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa de Simpson 3/8: 0.305042292581279

Erro: 0.01%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

13. Extrapolação por Richards

Estratégia de Implementação:

O arquivo contém a função `richards()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$, o intervalo proposto nos exemplos e a quantidade de subdivisões, e retorna a estimativa da integral. Além disso, o cálculo usa como base o método do trapézio.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em quatro linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente e a quarta recebe o número de subdivisões.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: $[1.0, 1.3]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa da extrapolacao de richards: 0.321485368419235

Erro: 0.0%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Subdivisões: 8

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa da extrapolacao de richards: 0.717356090899152

Erro: 0.0%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Subdivisões: 4

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa da extrapolacao de richards: 0.391557276777760

Erro: 0.0%

4. Problema 11.11:

Entrada:

Função: $\sin(x) \cdot (E)^{-x}$

Intervalo: $[0.0, 1.2]$

Subdivisões: 6

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa da extrapolacao de richards: 0.305067582025186

Erro: 0.0%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

14. Quadratura de Gauss

Estratégia de Implementação:

O arquivo contém a função `gauss()`, que realiza o cálculo necessário para o método. Ela tem como parâmetros a função $f(x)$ e o intervalo proposto nos exemplos, e retorna a estimativa da integral.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” de saída consiste em três linhas, a primeira recebe a função $f(x)$, a segunda e terceira recebem os valores X_0 e X_1 que compõem o intervalo a ser avaliado, respectivamente.

O arquivo de saída “out.txt” consiste em três linhas, a estimativa da integral do método, a integral correta e a porcentagem do erro.

Problema teste:

Para a realização do método, foram testados os problemas 11.1, 11.2, 11.3 da página 328 e o problema 11.1 da página 334 do livro Cálculo Numérico Neide Franco que foi disponibilizado pelo professor. Foram retirados os conjuntos de dados das tabelas das questões e gerado a estimativa da integral, a integral correta e a estimativa do erro.

1. Problema 11.1:

Entrada:

Função: $\sqrt{x}$

Intervalo: [1.0, 1.3]

Resultado Obtido:

Integral correta: 0.321485368419253

Estimativa da quadratura gaussiana: 0.321485695226566

Erro: 0.0%

2. Problema 11.2:

Entrada:

Função: $\cos(x)$

Intervalo: $[0.0, 0.8]$

Resultado Obtido:

Integral correta: 0.717356090899523

Estimativa da quadratura gaussiana: 0.717286669118528

Erro: 0.01%

3. Problema 11.3:

Entrada:

Função: $\sin(x)$

Intervalo: $[1.2, 1.6]$

Resultado Obtido:

Integral correta: 0.391557276777962

Estimativa da quadratura gaussiana: 0.391554944602397

Erro: 0.0%

4. Problema 11.11:

Entrada:

Função: $\sin(x) \cdot (E)^{-x}$

Intervalo: $[0.0, 1.2]$

Resultado Obtido:

Integral correta: 0.305067581870428

Estimativa da quadratura gaussiana: 0.305751491260426

Erro: -0.22%

Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

15. Considerações Finais

Após concluir este relatório de análise numérica, é evidente a importância dos métodos numéricos na resolução de problemas matemáticos. A implementação de algoritmos e a manipulação de dados permitem alcançar resultados precisos e eficientes. Os testes realizados confirmaram que a maioria das implementações utilizadas conseguiram encontrar estimativas de soluções satisfatórias e com pouca margem de erro.

No entanto, é necessário falar sobre os resultados inconsistentes no exercício 11.1 das derivadas de primeira ordem e os exercícios 11.1 e 11.2 das derivadas de segunda ordem, cujos valores das estimativas das derivadas centrada, progressiva e retardada foram consideravelmente diferentes da derivada correta.

Portanto, é necessário que futuras implementações sejam desenvolvidas considerando tais inconsistências. É essencial buscar constantemente aprimoramentos nos algoritmos utilizados e incorporar novas funcionalidades para garantir a eficácia dos métodos numéricos empregados. Somente assim será possível resolver problemas cada vez mais complexos e desafiadores com confiabilidade.